

番外編

静的解析のすゝめをやっていきます。

Javaの言語で書かれたファイルに対する静的解析の解説していききたいと思います。

最後にはたくさんの知識をインプットできるJavaの問題を紹介します。

問題に関しては解くかどうかは完全に自由です。時間余った方々のための知識補助みたいなもんです。

静的解析とはバイナリまたは実行ファイルをデコンパイルまたは逆アセンブリを行い、そのバイナリまたは実行ファイルが行う動作を解読することです。

デコンパイルとはコンピュータが実行できる機械語やバイトコードを人間が理解できるプログラミング言語のソースコードに変換する処理です。

Javaで例えると、classファイルに対してjavacの逆の操作を行なっているみたいな感じ
です。

メリット：

人間が理解しやすく可読性が逆アセンブリよりも高いこと

デメリット：

必要な情報が消える可能性がある

それに対して、逆アセンブリとはコンピュータが実行できる機械語やバイトコードをア
センブリ言語という鬼むず低レイヤ言語に変換する処理です。

メリット：

これが理解できれば基本的にファイルの動作はなんでもわかる

デメリット：

難しい・時間がかかる

初学者がやる内容では到底ないため、今回は番外編としてデコンパイルを持ってきました。概要を掴んでくれたら幸いです。

1つ目として次のようなコードを作成しました。

Main.javaのコード

```
1  import java.util.*;
2
3  /**
4   * あるシステムにログインIDとパスワードを入力してる感じのやつ
5   */
6  public class Main {
7      public static void main(String args[]) {
8          Scanner sc = new Scanner(System.in);
9          System.out.print(s: "Enter your ID: ");
10         String id = sc.nextLine(); // 入力受付し、入力されたものをidに代入
11         System.out.print(s: "Enter your Password: ");
12         int pass = sc.nextInt(); // 入力受付し、入力されたものをpassに代入
13         if (is_okay(id, pass)) {
14             System.out.println(x: "you made it!!!"); // クリア
15         } else {
16             System.out.println(x: "you failed to login"); // 失敗
17         }
18     }
19
20     /**
21      * ログインIDとパスワードが正しいかどうかを確認するクラスメソッド
22      * ログインIDはadminでパスワードは123
23      *
24      * @param id ログインID
25      * @param pass パスワード
26      * @return
27      */
28     public static boolean is_okay(String id, int pass) {
29         if (id.equals(anObject: "admin") && pass == 123) {
30             return true;
31         } else {
32             return false;
33         }
34     }
35 }
```

出力例 成功時：

```
● kdx:~/Downloads/s/instructor_
Enter your ID: admin
Enter your Password: 123
you made it!!!
```

まず、Javaファイルに対するデコンパイルがどれほど正確にできるのかを見せます。
(VSCodeのdecompiler拡張機能)

```
1  // Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).
2  import java.util.Scanner;
3
4  public class Main {
5      public Main() {
6      }
7
8      public static void main(String[] var0) {
9          Scanner var1 = new Scanner(System.in);
10         System.out.print("Enter your ID: ");
11         String var2 = var1.nextLine();
12         System.out.print("Enter your Password: ");
13         int var3 = var1.nextInt();
14         if (is_okay(var2, var3)) {
15             System.out.println("you made it!!!");
16         } else {
17             System.out.println("you failed to login");
18         }
19     }
20
21     public static boolean is_okay(String var0, int var1) {
22         return var0.equals("admin") && var1 == 123;
23     }
24 }
25
26
```

違いで言うと、変数名が違ふこととis_okayの条件分岐が少し違ふことが挙げられます。

変数名が違ふ理由としては `javac` はデフォルトでは行番号とソースファイル情報のみを生成し、ローカル変数名は含まれないからです。

(`-g` オプションをつけると `LocalVariableTable` が生成されて変数名も保持されるみたいです。)

条件分岐が違ふ理由はコンパイルするとgoto (特定の場所に飛ぶ) や条件分岐命令であるifeq (もし前の比較で値が一致したら) などのアセンブリ命令に変換されるため

です。もともと使用していたifとgotoやifeqは別のフローなので、デコンパイルされたコードも変わります。

また、次の2枚の画像を見てみてください。

```
5  */
6  public class Main {
    Run | Debug
7      public static void main(String args[]) {
8          String a = "";
9          String b = "I am from Kithub";
10         Scanner sc = new Scanner(System.in);
11         System.out.print(s: "Enter your ID: ");
12         String id = sc.nextLine(); // 入力受付し、入力されたものをidに代入
13         System.out.print(s: "Enter your Password: ");
```

```
1  // Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).
2  import java.util.Scanner;
3
4  public class Main {
5      public Main() {
6      }
7
8      public static void main(String[] var0) {
9          Scanner var1 = new Scanner(System.in);
10         System.out.print("Enter your ID: ");
11         String var2 = var1.nextLine();
12         System.out.print("Enter your Password: ");
13         int var3 = var1.nextInt();
14         if (is_okay(var2, var3)) {
15             System.out.println("you made it!!!");
16         } else {
17             System.out.println("you failed to login");
18         }
19     }
20
21
22     public static boolean is_okay(String var0, int var1) {
23         return var0.equals("admin") && var1 == 123;
24     }
25 }
26
```

使用していないString型の変数を定義しました。

コンパイルの特徴として実行時に不要なものを除去する最適化が行われます。なので、デコンパイルしても、classファイルにaやbに関する文字列情報がないため、デコンパイルしたコードに表示されません。

それにしてもこんなに復元できていいのかと思いますよね？

Java (≒Kotlin: Androidアプリ用コードを記述するため言語) はAndroidアプリでも使われています。

Androidアプリでよく見る (かもしれない) 拡張子のAPKも先ほど紹介したデコンパイルが可能です。

モバイルアプリ開発の子達が作ったアプリを静的解析...みたいなことができる訳です。

しかし!!! 実際のAndroidのアプリに対してやろうとするとめんどくさい問題に直面するはず。それは**難読化**されているという問題です。

難読化される影響:

1: クラス名やメソッド名、変数名が意味のない短い文字列になってしまう

2: 文字列が暗号化されてしまう

次のようなデコンパイル結果になってしまいます (簡単だからまだわかりやすい)

```
1 package com.a.a;
2
3 import java.util.*;
4
5 // クラスの名前が違う
6 public class a {
7
8     // adminが暗号化されてる
9     private static final String a = b.a("H4sIAAAAAAAAA0rNyclRSMsvyk1RBA0AAP//YqsDyhAAAAA=");
10    private static final int b = 123;
11
12    Run | Debug
13    public static void main(String[] args) {
14        Scanner sc = new Scanner(System.in);
15        System.out.print(s: "Enter your ID: ");
16        String a2 = sc.nextLine();
17        System.out.print(s: "Enter your Password: ");
18        int a3 = sc.nextInt();
19        if (a(a2, a3)) {
20            System.out.println(x: "you made it!!!");
21        } else {
22            System.out.println(x: "you failed to login");
23        }
24    }
25
26    // メソッドの名前が違う
27    public static boolean a(String str, int i) {
28        if (str.equals(b.a(a)) && i == b) {
29            return true;
30        } else {
31            return false;
32        }
33    }
34 }
```

このような難読化が施されていたとしても時間をかければ突破されてしまいます。

出ると予想される影響として機密情報や個人情報やマルウェアの拡散が考えられます。Android APKなどで調べると大量のAPK配布があります。大半がマルウェアだと思いますが、上記のような手段で解読されたものでちゃんと配布しているものもあります。

Androidで開発・リリースを考えている方は気をつけましょう。

また、アンドロイドユーザーは危ないAPKはダウンロードしないようにしてください。

問題です。以下の問題をダウンロードしてクリアしてください。

パスワードはcrackmes.oneです。(間違ってたらごめん)

<https://crackmes.one/crackme/693c48822d267f28f69b8518>

Javaのデコンパイラ+基本情報前半ぐらいの知識+Pythonの知識+セキュリティの勘があれば解けます。

これは初学者にとっては鬼難易度です。

いろんな知識が組み合わさってるな〜ぐらいの気持ちでAIに投げる

またはmaterialにある答えを見てみてください。いい経験になります(たぶん)

これはかなりリバエンよりの静的解析でしたが、どうでしたでしょうか？

解析系に興味があれば、CTFやCrackmes.oneで問題を解いてみてください。

続けていたら意味不明でも成長していくと思うので、頑張ってください。

参考資料

https://zenn.dev/agtech_tomoito/articles/34573115100de0